Plan de Pruebas

"SAPE CSS"

Fecha:[08/10/2026]

Tabla de contenido

Tabla de contenido **2**
1. Histórico de Revisiones **2**
2. Información del Proyecto **2**
Integrantes 2
3. Alcance de las Pruebas **3**
Excluido de esta iteración 3
4. Descripción General y Resumen de las Pruebas **3**
5. Objetivos de las Pruebas **4**
Detalle del Orden de Ejecución de los Módulos **4**
Tipos de Pruebas a Realizar **4**
Técnicas de Pruebas a Utilizar **5**
6. Roles y Responsabilidades **5**
7. Entorno y Configuración de las Pruebas **5**
Secuencia operativa de verificación 6
8. Calendarización de las Actividades de Pruebas **6**
Carta Gantt 6
9. Resumen de Riesgos **7**
10. Definición de Artefactos **7**
11. Condiciones de Aceptación para Cierre del Proceso **8**
12. Glosario **8**

1. Histórico de Revisiones

Versión	Fecha	Descripción / cambio	Autor
1.0	28/09/2026	Desarrollo	Felipe Catalan
1.1	05/10/2026	Riesgos	Felipe Catalan
1.2	08/10/2026	Ajuste de alcance, calendario y criterios de cierre	Equipo SAPE CSS

2. Información del Proyecto

Campo	Información
Organización	Duoc UC. Escuela de Informática y Telecomunicaciones
Sección	001D
Proyecto (nombre)	SAPE CSS
Fecha de inicio	21 septiembre 2026
Fecha de término	29 noviembre 2026
Patrocinador principal	Duoc UC
Docente	JUAN ANDRES HERNANDEZ ALIAGA

Integrantes

RUT	Nombre	Correo
21620250-3	Felipe Catalan	fe.catalanv@duocuc.cl
Por completar	José Valenzuela	jt.valenzuela@duocuc.cl
Por completar	Felipe Verdugo	feli.verdugo@duocuc.cl

3. Alcance de las Pruebas

Este plan define qué se valida, con qué datos y bajo qué criterios para la versión actual de SAPE CSS. Se basa en rutas,

controladores, pruebas automatizadas y configuración de integración continua disponibles en el repositorio.

Incluido

Área	Cobertura planificada
Acceso y cuenta	Registro, inicio y cierre de sesión, verificación de correo, recuperación/cambio de contraseña y perfil.
Gestión académica	Importación de archivos Excel, validación de formato, transacciones, prevención de duplicados por checksum y visualización de datos importados.
Paneles	Panel de administrador/director y paneles de profesor jefe y de asignatura; se valida entrega de datos según relación docente-curso.
Asistente pedagógico	Solicitud autenticada, cálculo agregado del curso, anonimización de nombres antes de la integración externa y restauración de la respuesta.
Datos sintéticos	Generación reproducible de archivos XLSX con hojas NOMINA y CALIFICACIONES, estructura, casos borde y datos ficticios.
Calidad técnica	Compilación frontend, estilo PHP con Pint y batería PHPUnit sobre PostgreSQL real, según CI.

Excluido de esta iteración

No se certifican servicios de terceros fuera del control del proyecto (disponibilidad, calidad pedagógica o costo de Gemini), envío real de WhatsApp, analítica predictiva/ML, pruebas de carga masiva ni despliegue productivo. Se prueban contratos, manejo de errores y simulaciones controladas cuando corresponde.

4. Descripción General y Resumen de las Pruebas

SAPE CSS es un monolito web Laravel 12 con Inertia y React. Centraliza la carga de nómina y calificaciones, permite a docentes visualizar cursos asignados y ofrece un asistente pedagógico que usa indicadores agregados del curso. Los perfiles internos son Administrador, Profesor Jefe y Profesor de Asignatura; el apoderado no posee inicio de sesión en el alcance actual.

Módulo	Pruebas principales	Prioridad
Autenticación y perfil	Flujos Breeze, protección de rutas y actualización de credenciales.	Alta
Importación Excel	Tipo de archivo, importación atómica, trazabilidad y deduplicación por SHA-256.	Crítica
Panel profesor	Curso y calificaciones visibles solo para el docente relacionado.	Alta
Panel administrador	Enrutamiento y presentación del tablero institucional.	Media
Chat pedagógico	Autorización, anonimización y respuesta controlada de proveedor externo.	Crítica

Generador sintético	Formato SIGE, bordes, falsedad de datos y repetibilidad por semilla.	Alta

5. Objetivos de las Pruebas

- Confirmar que los flujos críticos entregan el resultado esperado y protegen las rutas que requieren autenticación.
- Comprobar la integridad de la importación académica, incluyendo rechazo de formatos inválidos y ausencia de duplicación al repetir un archivo procesado.
- Verificar que el chatbot no transmite nombres de estudiantes al servicio externo y que la respuesta puede restaurarse para el usuario autorizado.
- Asegurar que los archivos sintéticos son estructuralmente compatibles, reproducibles y no contienen datos reales.
- Mantener una puerta de calidad repetible en CI: build del frontend, análisis de estilo y pruebas PHPUnit contra PostgreSQL.

Detalle del Orden de Ejecución de los Módulos

Orden	Bloque	Dependencia / razón
1	Preparación del entorno	Base para ejecutar build, migraciones, datos controlados y pruebas.
2	Autenticación y roles	Habilita los flujos y pantallas que demandan usuario autenticado.
3	Generador e importación	Entrega datos académicos para paneles y chatbot.
4	Paneles docentes y administrador	Consume relaciones de cursos, estudiantes y calificaciones.
5	Chat pedagógico y privacidad	Usa curso cargado y valida el límite de datos enviado al proveedor.
6	Regresión y CI	Repite las comprobaciones automáticas antes de cerrar la iteración.

Tipos de Pruebas a Realizar

Tipo	Aplicación
Unitarias	Privacidad/tokenización y reglas aisladas.
Integración	Controlador, base PostgreSQL, importación Excel, transacciones y llamadas HTTP simuladas.
Funcionales	Rutas, formularios, navegación por rol y visualización de información.
Regresión	Batería PHPUnit existente más ejecución completa del pipeline CI.
Seguridad y privacidad	Autenticación, middleware y verificación de que nombres no cruzan al proveedor de IA.

Compatibilidad de datos	Estructura XLSX con NOMINA/CALIFICACIONES y catálogos por nivel.

Técnicas de Pruebas a Utilizar

Técnica	Uso en SAPE CSS
Partición de equivalencia	Archivos válidos XLSX/XLS/CSV frente a extensiones no permitidas; mensajes válidos frente a ausentes.
Valores límite y casos borde	Notas bajo 4,0, retiros, asignaturas conceptuales, eximición y cursos de 3°/4° medio.
Caja negra	Validación de respuestas HTTP, redirecciones, mensajes de éxito/error y props de Inertia.
Caja blanca	Cobertura de ramas de anonimización, tokenización, idempotencia y errores transaccionales.
Doble de prueba	Http::fake para Gemini: sin costo, sin salida de datos y con respuestas deterministas.
Regresión automatizada	PHPUnit y Pint ejecutados en CI con entorno reproducible.

6. Roles y Responsabilidades

Rol	Responsabilidades
Responsable QA / integrante asignado	Planificar, preparar datos, ejecutar casos manuales, registrar evidencias y comunicar defectos.
Desarrollador/a	Corregir defectos, aportar pruebas automatizadas y revisar la regresión asociada.
Revisor/a técnico/a	Validar cambios, calidad de código y estado del pipeline CI.
Docente / patrocinador	Resolver criterios funcionales, revisar evidencias y autorizar el cierre académico.

7. Entorno y Configuración de las Pruebas

Componente	Configuración requerida
Aplicación	Laravel 12, Inertia 2 y React 18; dependencias restauradas con Composer y npm.
PHP	PHP 8.3 con pdo_pgsql, pgsql, fileinfo, zip, intl y gd.
Node	Node.js 22; ejecutar npm ci y npm run build antes de las pruebas de vistas.
Base de datos	PostgreSQL 16. Base de pruebas sape_test; usuario admin_db. No usar SQLite para el set de regresión.
Contenedores	Docker Desktop operativo y servicio PostgreSQL disponible antes de ejecutar PHPUnit.
Datos	Fixtures/datos sintéticos. Nunca usar información personal real en pruebas ni en llamadas de IA.

IA externa	Clave configurada únicamente en entorno controlado; Http::fake para prueba automática.

Secuencia operativa de verificación

Desde la carpeta **sape-css**:

1) npm ci y npm run build;

2) ./vendor/bin/pint --test;

3) php artisan test.

Esta secuencia replica el orden del flujo CI y evita fallas de vistas por manifiesto Vite ausente.

8. Calendarización de las Actividades de Pruebas

La calendarización se organiza por sprint: el proyecto se desarrolla en 5 sprints de 2 semanas entre el 21 de septiembre y el 29 de noviembre de 2026 (S1: 21 sep – 4 oct; S2: 5 – 18 oct; S3: 19 oct – 1 nov; S4: 2 – 15 nov; S5: 16 – 29 nov). Las pruebas se ejecutan dentro de cada sprint y los casos específicos se definen al planificar cada uno, a partir de las historias seleccionadas en Trello. Por eso, la asignación de historias desde el sprint 3 es tentativa y se confirma en cada planificación.

Actividad	Fechas	Sprint	Responsable	Resultado
Preparar ambiente, CI y datos sintéticos (INDG-25, 26, 16)	21 sep – 4 oct	S1	QA / Dev	Ambiente y pipeline operativos; datos controlados.
Probar autenticación, roles e importación ETL (INDG-24, 15)	21 sep – 4 oct	S1	QA / Dev	Evidencia de acceso por rol y de carga correcta.
Probar privacidad, chatbot, integridad del ETL y vista móvil (INDG-27, 17, 30, 29)	5 – 18 oct	S2	QA / Dev	Payload anonimizado, rollback del ETL y evidencia en pantallas móviles.
Probar buscador directivo y redactor de informes (INDG-21, 18)	19 oct – 1 nov	S3	QA / Dev	Registro de casos; anonimización y revisión humana verificadas.
Probar detección de riesgo, dashboard y mapa térmico (INDG-20, 22, 23)	2 – 15 nov	S4	QA / Dev	Indicadores explicables y casos borde validados; matriz de visibilidad por rol.
Probar mensajería a apoderados, rendimiento y arquitectura (INDG-19, 28, 31)	16 – 22 nov	S5	QA / Dev	Enlace de WhatsApp revisable; latencia medida (meta ≤ 2 s).
Regresión y revisión de defectos al cierre de cada sprint	Últimos 2 días de cada sprint	S1 – S5	Dev / QA	Pipeline en verde y defectos resueltos o aceptados.
Cierre y consolidación	23 – 29 nov	S5	QA / Docente	Informe final y aprobación.

Carta Gantt

Actividad	S1	S2	S3	S4	S5
Preparación / CI / datos	X				
Autenticación / ETL	X				
Privacidad, chatbot, ETL transaccional, vista móvil		X			
Buscador y redactor de informes			X		
Riesgo, dashboard y mapa térmico				X	
Mensajería, rendimiento y arquitectura					X
Regresión por sprint	X	X	X	X	X
Cierre					X

9. Resumen de Riesgos

Riesgo	Magnitud	Ocurrencia	Mitigación	Contingencia
PostgreSQL/Docker no disponible	Alto	Media	Verificar Docker y conexión antes de iniciar; documentar comandos de preparación.	Reprogramar ejecución automatizada; conservar evidencia manual sin declarar cierre.
Datos académicos reales expuestos	Alto	Baja	Usar solo datos sintéticos y revisar payload de IA con pruebas HTTP simuladas.	Detener prueba, revocar credenciales y registrar incidente.
Servicio Gemini no disponible o cambia	Significativo	Media	Aislar con Http::fake; validar contrato y error controlado.	Ejecutar pruebas sin red y marcar integración real como pendiente.
Archivo Excel con estructura distinta	Significativo	Media	Validar extensión, contrato de hojas/columnas y ejecutar con muestras controladas.	Rechazar archivo, registrar error y solicitar corrección de origen.
Regresión visual tras cambios frontend	Moderado	Media	Compilar Vite y realizar recorrido manual por rol en navegador soportado.	Revertir cambio funcional en la rama de trabajo y abrir defecto.
Duplicación por reimportación	Alto	Baja	Probar checksum e idempotencia antes de aceptar la carga.	Detener cargas, restaurar desde respaldo y revisar bitácora.

10. Definición de Artefactos

Artefacto	Descripción	Responsable
Plan de pruebas	Este documento: alcance, estrategia, entorno, calendario, riesgos y cierre.	QA
Casos y evidencia	Matriz de casos con ID, precondición, datos, pasos, esperado, obtenido, estado y capturas/logs.	QA
Reporte de defectos	Registro de ID, severidad, prioridad, pasos de reproducción, evidencia, responsable y estado.	QA / Dev
Reporte de ejecución	Resultado de npm build, Pint y PHPUnit; fecha, ambiente y responsable.	QA / CI
Archivos XLSX sintéticos	Datos de prueba generados con semilla y contrato NOMINA/CALIFICACIONES. No se versionan como evidencia final.	QA / Dev
Informe de cierre	Resumen de cobertura, defectos abiertos/aceptados y decisión de cierre.	QA / Docente

11. Condiciones de Aceptación para Cierre del Proceso

- 100% de los casos de prioridad crítica y alta ejecutados y aprobados: autenticación, importación, panel de profesor, privacidad del chatbot y generación sintética.
- Las verificaciones automatizadas del flujo CI finalizan correctamente: compilación frontend, Pint y PHPUnit contra PostgreSQL.
- No existen defectos abiertos de severidad crítica o alta sin una decisión formal y documentada del responsable funcional/docente.
- Los defectos de severidad moderada pueden permanecer solo si tienen plan de corrección, impacto conocido y aceptación explícita. Los de severidad baja se registran para la próxima iteración.
- Existe evidencia reproducible de la ejecución: versión/commit probado, fecha, entorno, resultado y responsable.
- No se incluyen datos personales reales en los artefactos de prueba ni se han enviado nombres de estudiantes a la integración de IA.

12. Glosario

Término	Definición
CI	Integración continua: proceso automático que compila y verifica la aplicación en cada cambio relevante.
Checksum	Huella SHA-256 utilizada para detectar que un archivo ya fue importado.
Inertia	Adaptador que permite servir páginas React desde controladores Laravel sin construir una API REST separada.
PHPUnit	Marco de pruebas automatizadas utilizado por el proyecto.
Pint	Herramienta de verificación de estilo de código PHP.
Tokenización	Reemplazo temporal de nombres por marcadores como [ALUMNO_1] antes de enviar texto a un proveedor externo.
XLSX	Formato de libro Excel usado para nómina y calificaciones.
Caso de prueba	Conjunto de precondiciones, datos, pasos y resultado esperado para validar un requisito.
Defecto	Comportamiento observado que no cumple un requisito, expectativa acordada o criterio de aceptación.